

Implementation Plan

Quantum-Train Classifiability-Predicted Ablation for a 2D Burgers' Physics-Informed Neural Network

mesh.applied

July 2026

Quantum-Train generates the weights of a classical neural network from a parameterized quantum circuit (PQC), then discards the circuit and deploys only the classical weights. Its own necessity proof does not cover circuits with one-dimensional or geometrically local structure, where a classical tensor network already reproduces the same computation. This plan implements a test of which side of that boundary a Quantum-Train weight generator falls on when used for a physics-informed neural network (PINN) solving the two-dimensional Burgers' equation: classify the circuit, predict the outcome, then run a matched-parameter-count ablation against two classical generators. It fixes who builds what, which tool does which job, and where every technique and number comes from. Every library and repository listed below was checked directly against its own documentation or source before being included.

1 Scope and division of labor

Two people, one paper, together as mesh.applied. Janos owns the quantum weight generator itself: making it a genuine compression scheme and getting it to converge. Petya owns the classification step, both classical baseline generators, and the ablation that compares all three.

Ph.	Owner	Deliverable	Where
0	Petya	RFF dequantization warm-up figure	experiments/phase0_ rff_warmup/
1a	Janos	Corrected weight generator (see status note below) trained to convergence	src/qt_pinn/qnn_ generator.py
1b	Petya, starts immediately	Circuit classification (independent of 1a); classical baselines sized once 1a's final parameter count is fixed	src/qt_pinn/ classification.py, src/qt_pinn/ baselines/
2	Petya	Ablation table: residual, generalization, stability, all three generators	experiments/phase2_ ablation/
3	Both	Technical note, reconciled against Janos's trained results	research/

Status, first pass reviewed. Janos's first implementation (6 qubits, 2 entangling layers, per-qubit $\langle Z_i \rangle$ readout) has two open problems that make it, currently, not a compression scheme and not yet converged. First, the qubit readouts are mapped to the target network's 418 weights through a learned, fully dense classical linear layer, which has 2,926 parameters on its own, against a 36-parameter quantum circuit: the generator as a whole is about seven times larger than what it generates, the reverse of the compression Liu et al. report (6,690 classical parameters compressed to 728 total, [1]). Liu et al. do not use a learned dense layer for this step at all: their mapping from measured probabilities to weights is a fixed, largely non-trainable formula (a bounded transformation of pairs of basis-state probabilities, with a single trainable scale parameter), so essentially all of their 728 reported parameters are the quantum circuit's own rotation angles. The fix is architectural, not a training fix: replace the dense projection with a small, mostly fixed mapping of this kind, so the generator's parameter count is dominated by the quantum circuit as intended. Second, independent of that fix, the current trained runs do not fit their own initial condition at $t = 0$, let alone the full PDE. Both are Phase 1a. This gates Phase 2: the ablation is only meaningful once the quantum side is an actual compression scheme, and only informative once it converges.

Janos's repository (<https://github.com/JanosMozer/qnn-burgers-eq>) had a first working push at the time of writing, so Phase 1b's classification does not have to wait on 1a landing; the classical baselines, however, need 1a's corrected parameter count before they can be sized to match it.

2 Technical stack

Role		Tool	Note
Quantum simulation	circuit	PennyLane, <code>default.qubit</code> at small scale, <code>lightning.gpu</code> if scale requires it	Janos's current 6-qubit circuit runs on PennyLane's CPU <code>default.qubit</code> device, appropriate at this size. <code>lightning.gpu</code> runs on NVIDIA's cuQuantum SDK and requires an SM 7.0 (Volta) or newer GPU; confirmed against the official device page, https://docs.pennylane.ai/projects/lightning/en/stable/lightning_gpu/device.html , and only becomes necessary if the 1a fix increases qubit count.
Circuit gradients		PennyLane adjoint method	$O(P)$ -time gradient computation for P parameters, based on the reverse-mode technique of Jones and Gacon [5].
Quantum-classical bridge		<code>pennylane.qnn.TorchLayer</code>	Wraps a PennyLane QNode as a PyTorch <code>nn.Module</code> layer, confirmed at https://docs.pennylane.ai/en/stable/code/api/pennylane.qnn.TorchLayer.html . Used to embed the PQC generator inside the same training loop as the classical PINN.
Classical training framework	training	PyTorch, <code>torch.optim.Adam</code> then <code>torch.optim.LBFGS</code>	Matches the two-stage optimization already used in Janos's design.
Classical-simulability classification	clas-	PennyLane <code>classical_shadow</code> and <code>shadow_expval</code>	Implements the randomized-measurement protocol of Huang, Kueng, and Preskill [4], the same protocol Cerezo et al. [2] use to check whether a circuit's measurement statistics can be reproduced classically after a data-acquisition phase. Confirmed at https://docs.pennylane.ai/en/stable/code/api/pennylane.classical_shadow.html .
Tensor-network classical baseline		<code>quimb</code>	Python library for tensor networks including matrix product states (MPS), actively maintained, supports PyTorch and JAX backends via <code>autoray</code> . Confirmed at https://github.com/jcmgray/quimb .
Random-Fourier-features classical baseline (Phase 0)		Hand-written, not <code>sklearn</code>	Scikit-learn's <code>RBFSampler</code> implements random Fourier features for a Gaussian kernel, a different construction from the integer-frequency, re-weighted-distribution setup in Sweke et al. Section 4.5 [3]. The warm-up needs the latter, so this piece is written by hand rather than imported.
Reference implementation	imple-	Official Quantum-Train repository	https://github.com/Hon-Hai-Quantum-Computing/QuantumTrain . Its primary backend is TorchQuantum with Qiskit (0.45.0), not PennyLane; the README states a PennyLane and Lightning-GPU version was also tried, "although some details may differ." Used here as a reference for the weight-mapping mechanism, not as a dependency.
Compute		RTX 5090, 32GB VRAM	Sized against Janos's own estimate for 8 to 14 qubits, comfortably inside <code>lightning.gpu</code> 's memory bounds at that qubit count.

3 Phase 0: dequantization warm-up

Goal: confirm, with our own numbers, the failure boundary that Sweke et al. [3] prove for random Fourier features (RFF) dequantizing a parameterized quantum circuit (PQC) regressor.

- Build a data-reuploading PQC regressor whose frequency re-weighting distribution is uniform or product-induced across its Fourier spectrum, the construction in [3], Section 4.5, for which their

Lemma 3 proves the RFF feature count M must scale super-polynomially in the problem dimension d .

- Build the matching hand-written RFF regressor (see stack table above).
- Sweep M against d and compare the empirical scaling to Lemma 3’s prediction.

Deliverable: one figure showing measured $M(d)$ against the theoretical bound, written up as a short, self-contained section usable even if later phases stall.

4 Phase 1a and 1b: fixing the generator, classification, and classical baselines

Janos’s task (1a). Replace the dense classical projection layer with a small, mostly fixed mapping from measured qubit values to target weights, in the style of Liu et al. [1], so the generator’s parameter count is genuinely dominated by the quantum circuit rather than a learned classical layer. Debug convergence: the current runs do not fit their own initial condition. The circuit itself, a strongly-entangling ansatz with circular (range-based, non-nearest-neighbor) CNOT connectivity, the Fourier feature front end, and the physics-informed loss (PDE residual plus initial and boundary conditions) optimized with Adam then L-BFGS, do not need to change for this fix.

Petya’s task (1b), classification starts immediately, baselines wait on 1a.

- Classify the ansatz’s causal light cone using `classical_shadow` and `shadow_expval`, following the protocol in Cerezo et al. [2]. This does not depend on the projection-layer fix and produces the a priori prediction: inside the classically simulable regime, expect no measurable advantage from the quantum generator; outside it, the outcome is open.
- Once 1a fixes the generator’s true parameter count, build the two classical baseline generators at that matched count: a random low-rank projection, and an MPS generator in `quimb` with bond dimension set by the classification step.

Deliverable: corrected, converged quantum generator (1a); classification verdict, stated prediction, and both classical baselines implemented and unit-tested against the corrected generator’s parameter count (1b).

5 Phase 2: ablation

Gated on 1a: requires the corrected, converged quantum generator. Train all three generators, quantum, low-rank, and MPS, under identical conditions: same PINN backbone, same collocation points, same optimizer schedule, on the same 2D Burgers’ instance, using Janos’s corrected and trained checkpoint for the quantum arm.

Metrics: PDE residual error, initial and boundary condition loss, generalization error on a held-out spatiotemporal region, and variance across at least five random seeds.

Deliverable: one comparison table, plus a direct statement of whether the empirical result matches the Phase 1 prediction.

6 Phase 3: writeup

Assemble the results into a short technical note covering the classification verdict, the ablation table, and their interpretation, citing and explicitly distinguishing the three close prior works already identified [6, 7, 8]. Reconcile the ablation against whatever Janos has trained by this point. Write a plain-language section explaining the physics and quantum-mechanics background used along the way, for readers who have not seen the rest of the project.

7 Resource index

Papers

Liu et al., Training Classical Neural Networks by Quantum Machine Learning	https://arxiv.org/abs/2402.16465
Cerezo et al., Does provable absence of barren plateaus imply classical simulability?	https://arxiv.org/abs/2312.09121
Sweke et al., Potential and limitations of random Fourier features for dequantizing QML	https://arxiv.org/abs/2309.11647
Huang, Kueng, Preskill, Predicting many properties of a quantum system from very few measurements	https://arxiv.org/abs/2002.08953
Jones, Gacon, Efficient calculation of gradients in classical simulations of variational quantum algorithms	https://arxiv.org/abs/2009.02823
Liu et al., Quantum-Enhanced Parameter-Efficient Learning for Typhoon Trajectory Forecasting	https://arxiv.org/abs/2505.09395
Zabihi, Montazeri, Suiker, Hybrid quantum-classical PINNs for nonlinear PDEs	https://arxiv.org/abs/2606.04679
Linghu, Dong, Wang, Trainable Photonic Measurement for Physics-Informed PDE Learning	https://arxiv.org/abs/2606.18713

Repositories

Janos's project repository	https://github.com/JanosMozer/qnn-burgers-eq
Official Quantum-Train reference implementation	https://github.com/Hon-Hai-Quantum-Computing/QuantumTrain
quimb (tensor networks, MPS)	https://github.com/jcmgray/quimb

Libraries and documentation

PennyLane	https://docs.pennylane.ai
PennyLane Lightning-GPU device	https://docs.pennylane.ai/projects/lightning/en/stable/lightning_gpu/device.html
<code>pennylane.qnn.TorchLayer</code>	https://docs.pennylane.ai/en/stable/code/api/pennylane.qnn.TorchLayer.html
<code>pennylane.classical_shadow</code>	https://docs.pennylane.ai/en/stable/code/api/pennylane.classical_shadow.html
PyTorch	https://pytorch.org

Techniques

Adjoint differentiation for quantum circuit gradients	Jones and Gacon [5]
Classical shadows (randomized measurement protocol)	Huang, Kueng, Preskill [4]
Light-cone-reduced classical simulation	Cerezo et al. [2]
Matrix product states as a classical weight-generator baseline	quimb documentation, and the tensor-network caveat in Liu et al. [1]

References

- [1] C.-Y. Liu, E.-J. Kuo, C.-H. A. Lin, S. Chen, J. G. Young, Y.-J. Chang, M.-H. Hsieh. Training Classical Neural Networks by Quantum Machine Learning. arXiv:2402.16465, 2024.
- [2] M. Cerezo, M. Larocca, D. García-Martín, N. L. Diaz, P. Braccia, E. Fontana, M. S. Rudolph, P. Bermejo, A. Ijaz, S. Thanasilp, E. R. Anschuetz, Z. Holmes. Does provable absence of barren plateaus imply classical simulability? Nature Communications 16, 7907, 2025. arXiv:2312.09121.
- [3] R. Sweke, E. Recio-Armengol, S. Jerbi, E. Gil-Fuster, B. Fuller, J. Eisert, J. J. Meyer. Potential and limitations of random Fourier features for dequantizing quantum machine learning. Quantum 9, 1640, 2025. arXiv:2309.11647.
- [4] H.-Y. Huang, R. Kueng, J. Preskill. Predicting many properties of a quantum system from very few measurements. Nature Physics 16, 1050-1057, 2020. arXiv:2002.08953.

- [5] T. Jones, J. Gacon. Efficient calculation of gradients in classical simulations of variational quantum algorithms. arXiv:2009.02823, 2020.
- [6] C.-Y. Liu, K.-C. Chen, Y.-C. Chen, S. Y.-C. Chen, W.-H. Huang, W.-J. Huang, Y.-J. Chang. Quantum-Enhanced Parameter-Efficient Learning for Typhoon Trajectory Forecasting. arXiv:2505.09395, 2025.
- [7] K. Zabihi, H. Montazeri, A. S. J. Suiker. Hybrid quantum-classical physics-informed neural networks for solving nonlinear PDEs: when and where hybridization is effective? arXiv:2606.04679, 2026.
- [8] J. Linghu, H. Dong, Y. Wang. Trainable Photonic Measurement for Physics-Informed PDE Learning. arXiv:2606.18713, 2026.